

Table of Contents

PROJECT OVERVIEW & OBJECTIVES.....	2
Project Objectives	2
SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC).....	3
Requirements Analysis	3
System Design.....	3
Implementation (Development)	3
Testing.....	3
Deployment.....	4
Maintenance.....	4
SYSTEM ARCHITECTURE DESIGN	5
Presentation Layer (Web / Mobile App Client)	6
Application Layer (Application Server)	6
Data Layer (Database)	6
Functional Requirements	6
Non-Functional Requirements	7
Prototype	8
Gantt Chart	11
Network Diagram.....	11
License.....	12
SDG Relevance	12
Conclusion.....	12
References	12

PROJECT OVERVIEW & OBJECTIVES

The **Smart Health Connect System** is designed as a Digital Public Good to address real-world gaps in healthcare accessibility, communication, and data management. The system creates a centralized, secure infrastructure connecting three primary user groups: Patients, Doctors, and Administrators. By digitizing core clinical processes—such as appointment scheduling, medical record tracking, prescription issuance, and user management—the platform reduces clinical administrative overhead and improves patient health tracking outcomes.

Project Objectives

- To apply formal Software Development Life Cycle (SDLC) methodologies to build a resilient healthcare tool.
- To establish a structured 3-Tier system architecture detailing the precise interaction between presentation, application, and database components.
- To create functional models mapping system behavior against core real-world healthcare needs.

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

To ensure systematic development, quality control, and scalable deployment, the Smart Health Connect System utilizes the **Waterfall SDLC Model**. Because healthcare systems require strict security boundaries, definite data processing standards, and predefined functional requirements, a phased, structured approach ensures each architectural stage is completely validated before technical execution begins.

The six phases of the development cycle are executed as follows:

Requirements Analysis

In this initial stage, the complete operational needs of all stakeholders (Patients, Doctors, and Administrators) are gathered and detailed. Core system functions are explicitly defined, such as patient registration, prescription generation, and data reports. These requirements form the baseline for the entire scope of the platform.

System Design

During the design phase, technical architecture and relational models are constructed. This involves mapping out the 3-Tier physical infrastructure and modeling the behavioral relationships of actors within the system boundaries using Use Case diagrams. User interfaces and core user workflows are also structured at this stage.

Implementation (Development)

With completed structural diagrams serving as blueprints, actual engineering begins. Database schemas are initialized, server-side business logic is compiled to process appointment routing, and front-end user client applications are constructed based on design specifications.

Testing

Once the system components are integrated, the software undergoes strict verification protocols. Functional testing validates that all paths outlined in the use case diagrams

function smoothly (e.g., confirming that a prescription created by a doctor is immediately viewable on the matching patient profile). Security audits check data access limits between roles.

Deployment

Following successful validation, the application code, server environment, and database structures are compiled and pushed to a secure staging environment or public server network. The GitHub repository serves as the core build pipeline tracker throughout this integration.

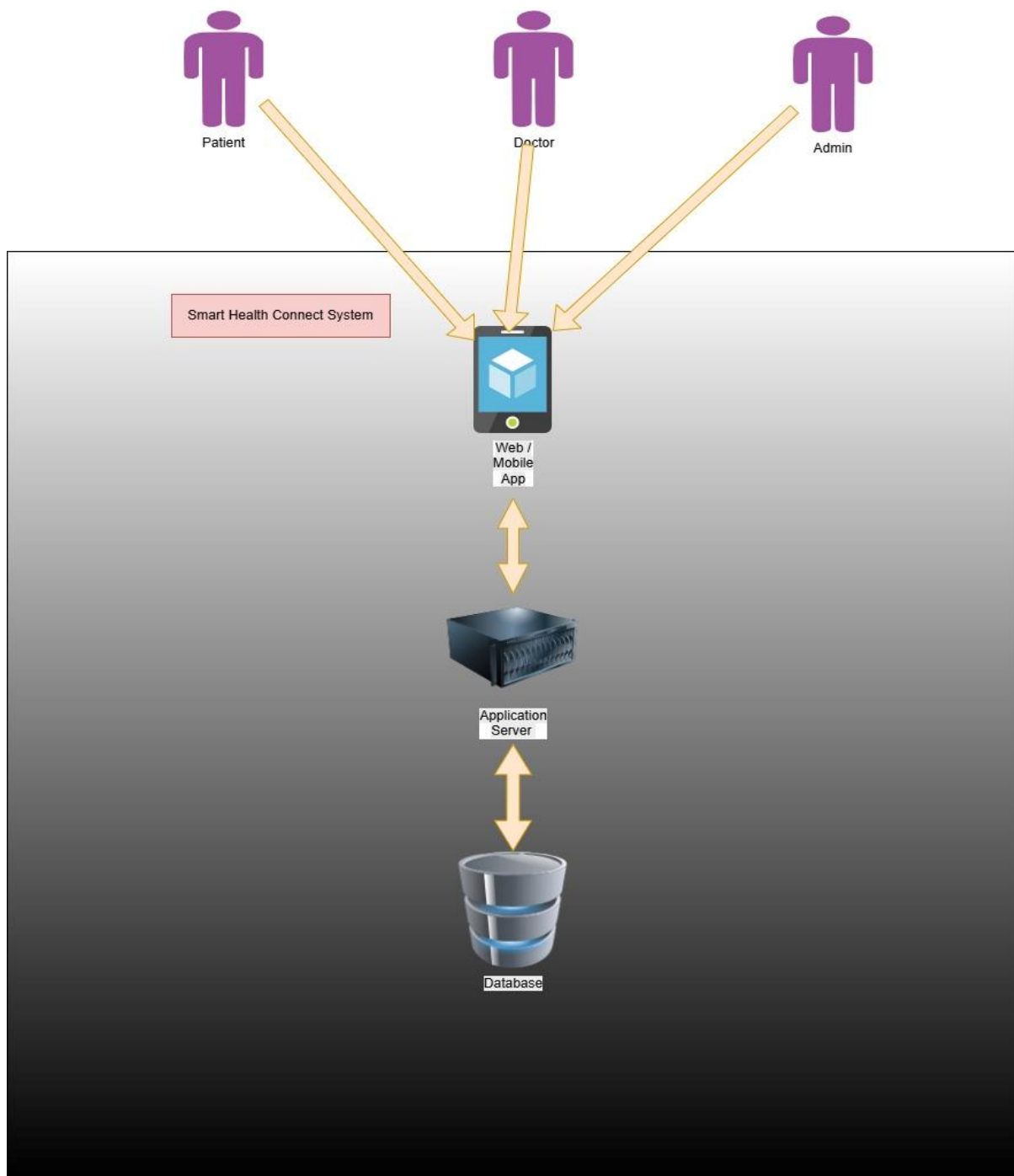
Maintenance

Post-deployment, the system is consistently kept up to date. This ongoing phase addresses system optimization updates, bug resolution, regular database backups, and security patch updates to safeguard confidential medical histories.

SYSTEM ARCHITECTURE DESIGN

As illustrated in **below**, the Smart Health Connect System is built using a decentralized, hierarchical **3-Tier Architecture**. This tier isolation ensures that user clients cannot directly access data structures without going through security protocols and business logic processing handled by the middle layer.

ARCHITECTURE DIAGRAM



Presentation Layer (Web / Mobile App Client)

This is the front-facing tier of the application that users directly interface with.

- **Actors Supported:** Patients, Doctors, and Administrators.
- **Function:** It displays data fields retrieved from the server and captures user input events (e.g., clicking a button to request an appointment or entering registration credentials). It communicates with the backend solely via standard application interface requests.

Application Layer (Application Server)

The center engine of the infrastructure that processes all operational commands and handles data traffic rules.

- **Function:** When a user executes an action on the app client, the request is routed here. The server runs specific business logic rules—such as verifying administrative security tokens, checking schedule conflicts before confirming an appointment, or filtering patient reports. It acts as a bridge, retrieving raw data from the database, formatting it, and passing clean output back to the presentation layer.

Data Layer (Database)

The secure database tier built for data persistence.

- **Function:** It is responsible for storing structural records, medical files, authentication credentials, and system logs. It only communicates with the application server layer, completely preventing outside actors from directly tampering with raw system storage.

Administrator

Functional Requirements

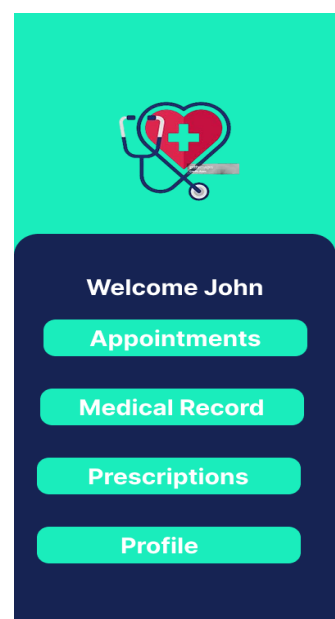
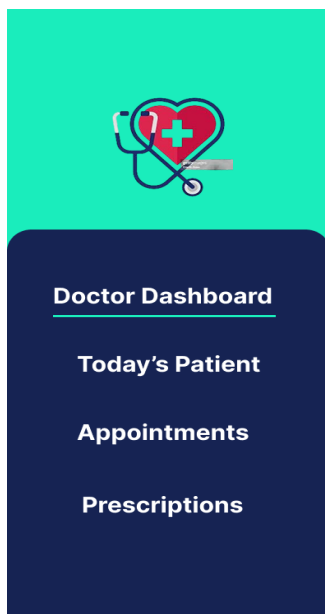
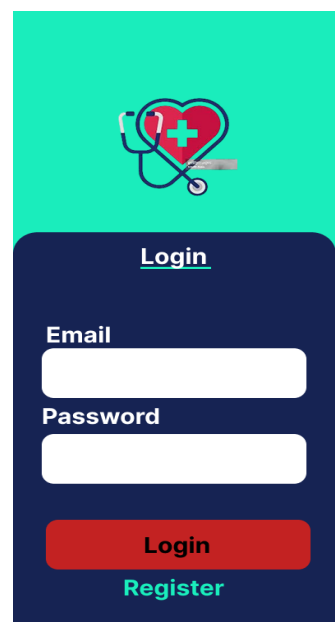
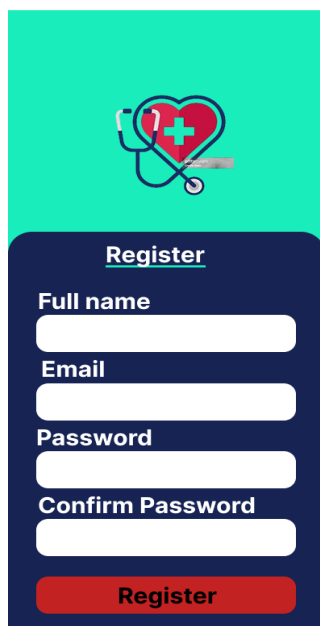
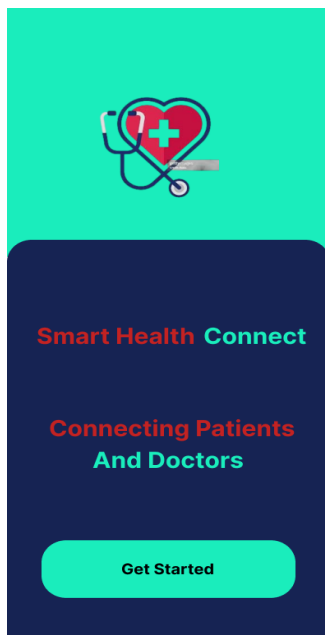
- User registration

- User login
- Course enrollment
- View course materials
- Upload learning materials
- Take quizzes
- View results

Non-Functional Requirements

- Security
- Reliability
- Fast response time
- Easy-to-use interface

Prototype





Doctor

Select

Date

Select

Time

Select

Book Appointment



Appointments

Dr. Kamara
12 June 2026
10:00 AM

Dr. Jabbie
20 June 2026
2:00 PM



Medical Records

Blood Pressure
120/80

Diagnosis
Malaria

Treatment
Medication



prescriptions

Paracetamol
2x Daily

Vitamin c
1x Daily



Profile

Name

Email

Phone

Edit Profile

Logout

1. EXECUTIVE SUMMARY

1.1 Project Overview

The Smart Health Connect System (engineered via the "Clinic Queue Pro" architecture) is an integrated desktop application designed to bridge the gap between healthcare administrative efficiency and user-centric patient care. Built using an advanced Python Object-Oriented Programming (OOP) framework, the system manages patient lifecycles from dynamic self-registration and secure authentication to real-time clinical queue sequencing and calendar allocation.

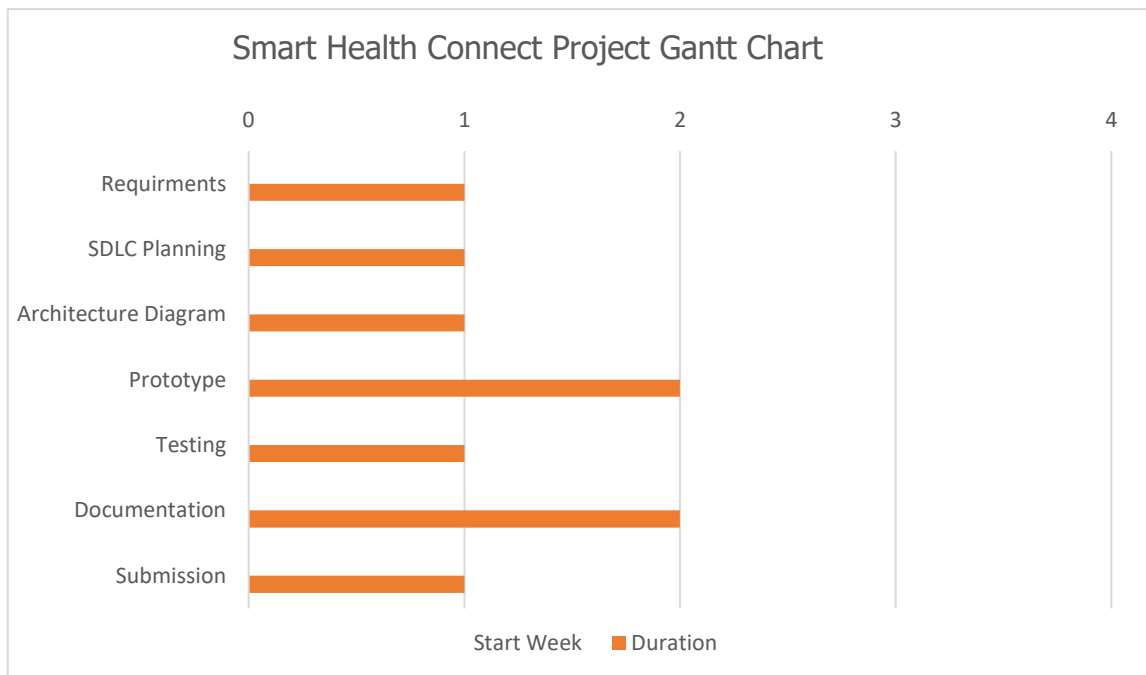
1.2 Problem Statement & Solution

Traditional clinical environments suffer from physical queue congestion, unpredictable wait times, and fragmented schedule logging. This systemic drag causes cognitive friction for both administrative staff and patients seeking urgent care.

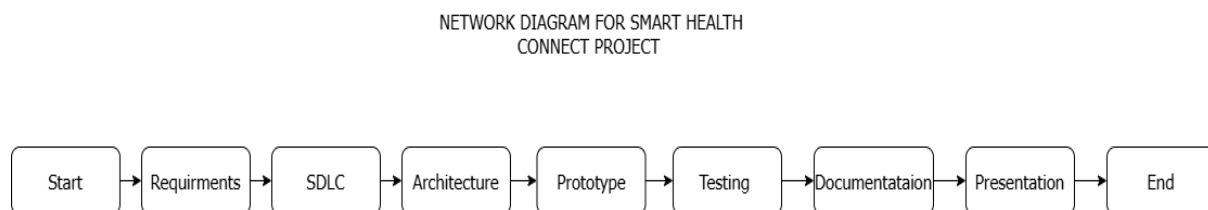
Clinic Queue Pro solves these bottlenecks by implementing an algorithmic, data-driven framework:

- Staff/Practitioner Node: Grants administrative, medical, and analytical views to track daily queue volumes, patient registration entries, and global performance metrics.
- Patient Node: Provides an autonomous portal for instant registration, secure credential hashing, historical medical record retrieval, and real-time live tracking of queue tokens. The prototype was developed using figma to demonstrate the user interface and navigation flow of the system.

Gantt Chart



Network Diagram



The network diagram illustrates the sequence of activities involved in the Smart Health Connect project. The project begins with requirements gathering and SDLC planning, followed by architecture design, prototype development, testing, documentation, presentation, and final completion.

License

This project uses the MIT License. The license allows users to use, modify, distribute, and share the software while maintaining attribution to the original creators.

SDG Relevance

Smart Health Connect supports Sustainable Development Goal 3 (Good Health and Well-Being) by improving access to healthcare services, facilitating appointment scheduling, and enhancing communication between patients and healthcare providers.

Conclusion

Smart Health Connect is a healthcare management system designed to improve the delivery of healthcare services. The project followed the Software Development Life Cycle (SDLC) and included system architecture design, prototype development, project planning, and repository management. The solution contributes to efficient healthcare management and supports improved health outcomes.

References

- Figma Documentation
- GitHub Documentation
- SDLC course materials
- Draw.io Documentation

